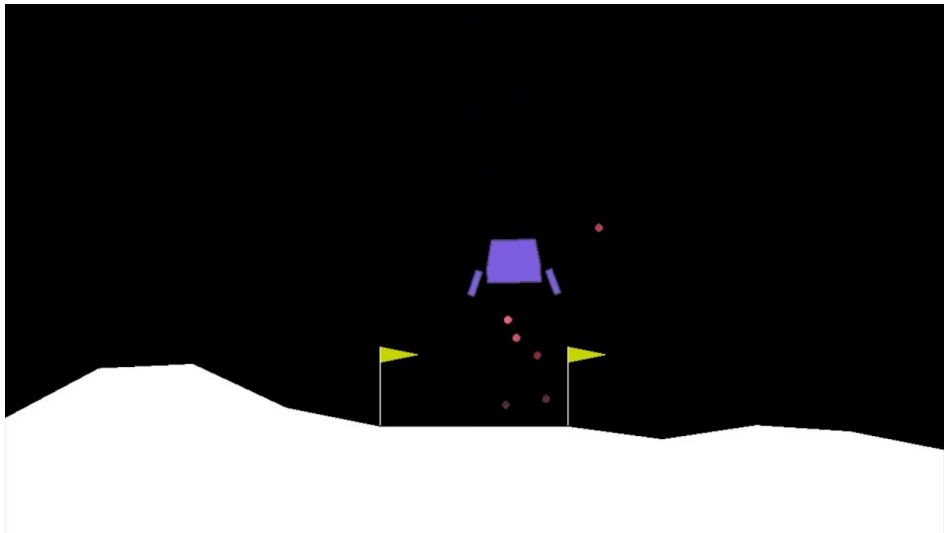
Solving The Lunar Lander Problem under Uncertainty using Reinforcement Learning

Yunfeng Xin, Soham Gadgil, Chengzhe Xu

Elia Generali

Corso di Studio in Intelligenza Artificiale (UniFi)

Introduction



"The ***main goal*** of the game is to direct the agent to the ***landing pad*** with as ***softly*** and ***fuel-efficiently*** as possible.
The state space is continuous as in real physics, but the action space is discrete."

Workspace

State Space

- x coordinate of the lander
- y coordinate of the lander
- v_x, the horizontal velocity
- v_y, the vertical velocity
- θ, the orientation in space
- v_θ, the angular velocity
- Left leg touching the ground (Boolean)
- Right leg touching the ground (Boolean)

Action Space

- do nothing
- fire left orientation engine
- fire right orientation engine
- fire main engine

Continuous ***Discrete***

Reward function

$$\begin{aligned}\textbf{Reward}(s_t) = &-100 * (d_t - d_{t-1}) - 100 * (v_t - v_{t-1}) \\ &- 100 * (\omega_t - \omega_{t-1}) + \text{hasLanded}(s_t)\end{aligned}$$

Where:

- dt**:** is the distance to the landing pad
- vt**:** is the velocity of the agent
- w_t**:** is angular velocity of the agent at time t.
- $hasLanded()$**:** is the reward function of landing, which is a linear combination of the boolean state values representing whether the agent has landed softly on the landing pad.

APPROACHES - Sarsa

Update Rule

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_t + Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

At any given state, the agent chooses the action with the highest Q-value:

Greedy Action

$$a = \underset{a \in \text{actions}}{\text{argmax}}\, Q(s, a)$$

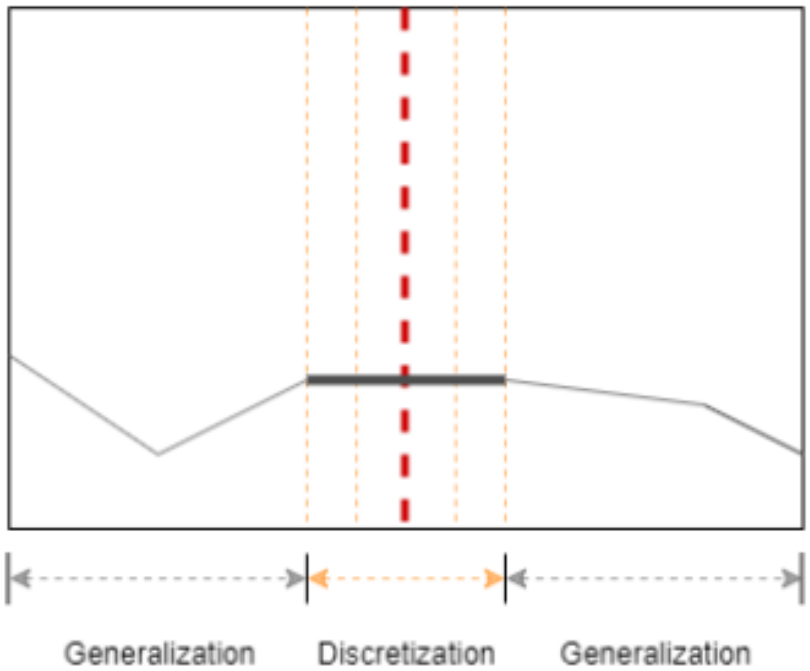
Q-Table

$s \backslash a$	a_1	$\dots$	a_m
s_1	$Q(s_1, a_1)$	$\dots$	$Q(s_1, a_m)$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
s_t	$Q(s_t, a_t)$	$\dots$	$\dots$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
s_{t+1}	$\dots$	$Q(s_{t+1}, a_{t+1})$	$\dots$
s_n	$Q(s_n, a_1)$	$\dots$	$Q(s_n, a_m)$

State Discretization

Define the discretization of the x coordinate at any state with the discretization step size set at 0.05 as:

$$d(x) = \min\left(\lfloor \frac{n}{2} \rfloor, \max\left(-\lfloor \frac{n}{2} \rfloor, \frac{x}{0.05}\right)\right)$$



APPROACHES - Deep Q-Learning

Update Rule

$$Q(s, a) = Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

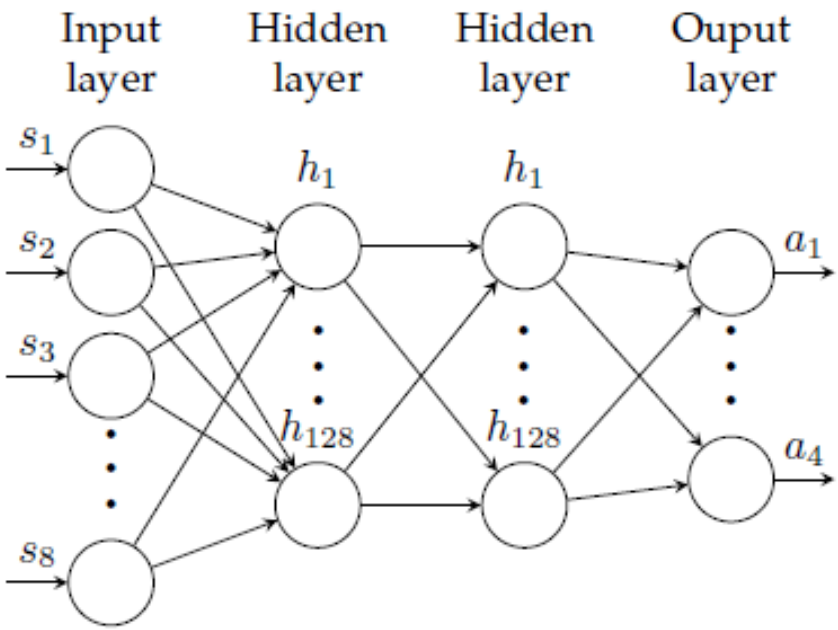
The optimal Q-value $Q^*(s; a)$ is estimated using the neural network with parameters θ. This becomes a regression task, so the loss function at iteration i is obtained by the temporal difference error:

Loss Function

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a)\sim\rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right]$$

where

$$y_i = \mathbb{E}_{s'\sim\mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \right]$$



Discretization Results

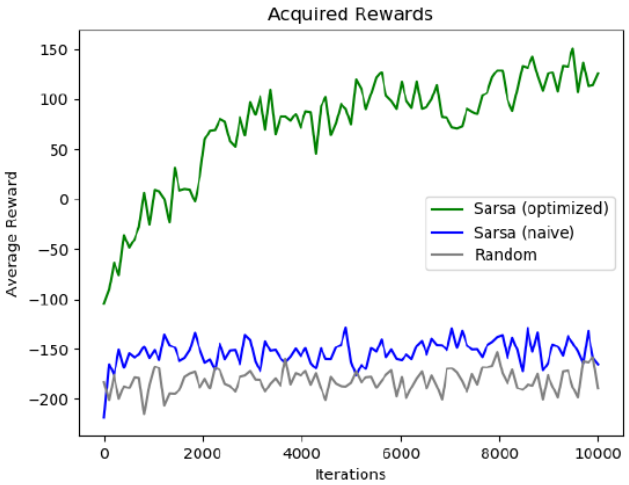


Fig. 5: Performance comparison of our state discretization and policy scheme (green), naive state discretization and policy scheme (blue), and random agent (grey).

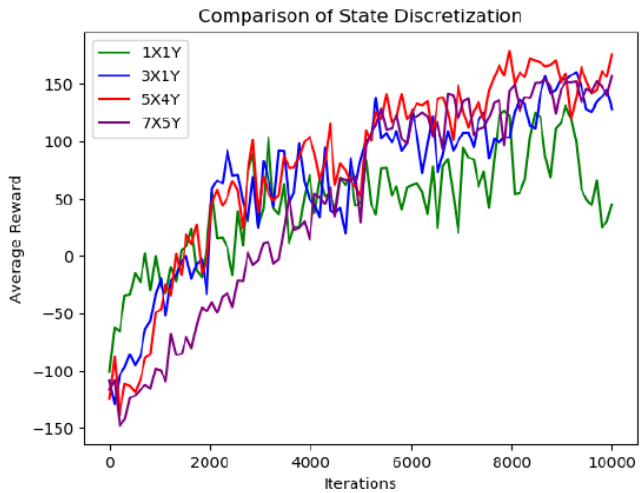


Fig. 6: Performance comparison of different state discretization schemes in Sarsa.

Introducing Additional Uncertainty

Noisy Observations

For each observation of x, we sample a random zero-mean Gaussian noise:

$$s \sim \mathcal{N}(\mu = 0, \sigma = 0.05)$$

and add the noise to the observation, so that the resulting random variable becomes:

$$\mathit{Observation}(x) \sim \mathcal{N}(x, 0.05)$$

We calculate the alpha vector of each action using one-step lookahead policy using the *Qvalues* from the original problem, and calculate the belief vector using the Gaussian probability density function.

Introducing Additional Uncertainty

Random Engine Failure

We introduce action failure in the lunar lander. The agent takes **the action provided 80% of the time**, but **20% of the time the engines fail** and it takes no action even though the provided action is firing an engine.

Random Force

We apply a random force each time the agent interacts with the environment and modify the state accordingly. **The force is sampled from a Gaussian distribution** for better simulation of real-world random forces.

Sarsa Results

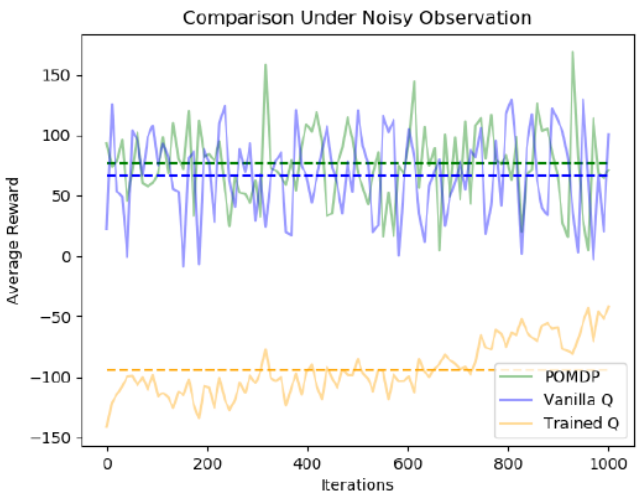


Fig. 7: Comparison of different agent performance under noisy observations.

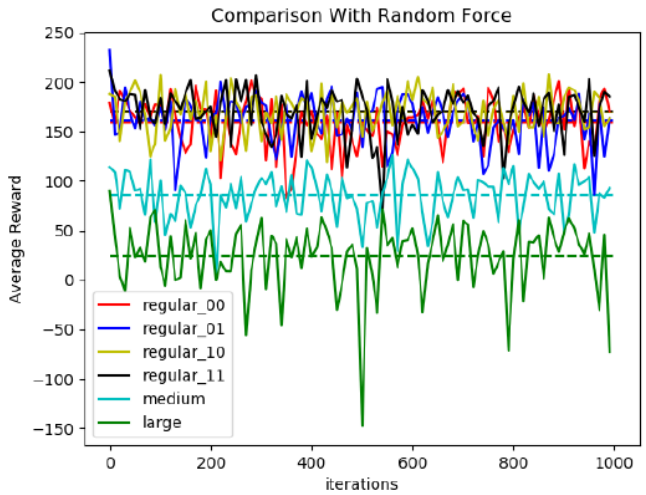


Fig. 8: Comparison of agent performance under different random forces.

DQN Results

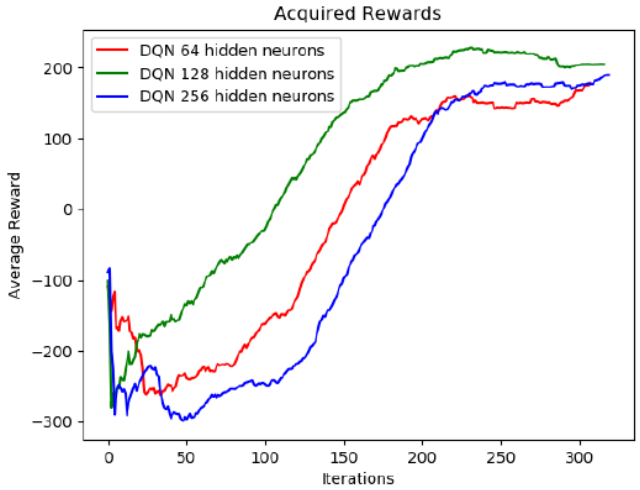


Fig. 9: Average reward obtained by DQN agent for different number of hidden neurons

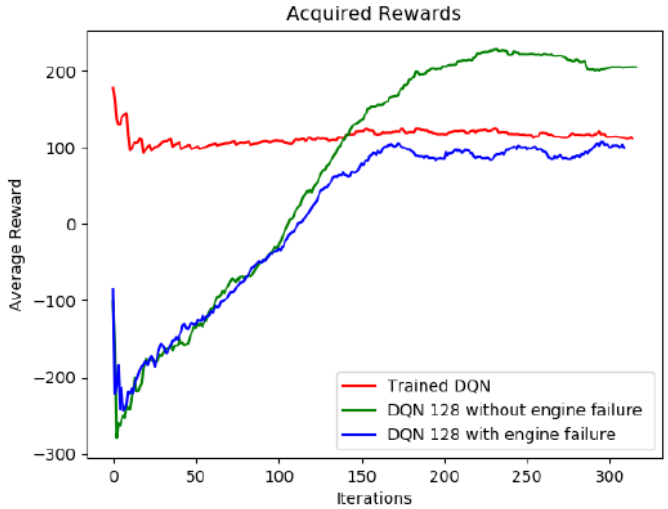


Fig. 10: Comparison of DQN agents under engine failure

Conclusions & Future Work

Experimental Results

- **Original Environment:** Both **Sarsa** and **DQN** agents perform well on the standard Lunar Lander problem.
- **Robustness under Uncertainty:**
  - **Sarsa** successfully handles random forces but **fails with noisy observations** due to its inability to generalize the altered state space.
  - **DQN** effectively manages engine failures.
  - **POMDP** achieves positive average rewards under noisy observations by leveraging belief vectors to model state distributions.

Future Work

- Combine different uncertainties simultaneously rather than testing them in isolation.
- Merge on-policy and off-policy networks to leverage speed and utilize continuous action spaces.

Thank you for your attention.